



Studying structural properties and exact models for solving the Job Sequencing and Tool Switching Problem

Shankar Narayanan

Internship report

Advisors:

Prof. Rafael Colares Prof. Annegret Wagler

Supervisor:

Dr. Renaud Chicoisne

Laboratoire d'Informatique, de Modélisation et d'Optimisation des Systèmes (LIMOS)

July 20, 2026

Abstract

The Job Sequencing and Tool Switching Problem (SSP) seeks an ordering of jobs on a single flexible machine, together with a tool-loading policy for a capacitated magazine, that minimises the number of tool switches. For a fixed order the loading is solved exactly by a classical greedy rule, so the difficulty lies in the ordering, and the problem is NP-hard. In practice the SSP is solved by a two-phase decomposition: group the jobs into as few magazine-fitting batches as possible, then sequence the batches. This report asks two questions about that standard method. *How wrong can the decomposition be?* A structural theory answering this question is developed in Section 3, which is withheld from the present draft while the author completes an independent verification of its proofs; it will be restored in the next revision. *Can the same grouping view solve the SSP exactly?* Here the obstacle is one number, the coverage bound $|U| - b$: we prove that every natural configuration relaxation is stuck at it, and a complete benchmark of five exact methods on 1,421 standard instances shows the suites split roughly in half between instances where this bound equals the optimum—easy for every method—and instances where no current method has traction.

Contents

1	Introduction	1
2	Preliminaries	3
2.1	The Job Sequencing and Tool Switching Problem	3
2.2	The tool-loading oracle	4
2.3	Grouping, sequencing, and the two-phase heuristic	5
2.4	The configuration-GTSP lens	5
2.5	Two structural lower bounds	6
2.6	Switch-cost convention	6
2.7	Related work	7
3	Structural analysis of the heuristic gap	8
4	Exact methods	8
4.1	Compact formulations and the coverage relaxation	8
4.2	A branch-and-Benders-cut algorithm	8
4.3	Position-indexed formulations	10
5	Computational study	12
5.1	Experimental design	13
5.2	Results	14
5.3	The competitiveness of configuration branch-and-price	16
6	Conclusions	16
6.1	Summary	16
6.2	Beyond the project horizon	16
7	Remaining work	17
A	Proofs	18

1 Introduction

Motivation. A flexible manufacturing cell built around a single computer-numerically-controlled machine can produce a wide variety of parts because it draws on a large library of cutting tools.

The machine, however, holds only a limited number of tools at once, in a magazine of capacity b ; producing the full part mix therefore requires tools to be exchanged between jobs. Each exchange interrupts production while the magazine is reconfigured, so tool switches are the dominant controllable cost in such cells [11]. When the set of tools needed by each job is fixed, two decisions remain: in what *order* to process the jobs, and which tools to *keep or evict* at each step. The Job Sequencing and Tool Switching Problem (SSP) is the joint optimisation of these two decisions so as to minimise the total number of switches. The same structure—a capacitated store serving set-valued requests that may be reordered—recurs beyond machining, for example in printed-circuit-board assembly and in warehouse order picking, so the methods developed here are not specific to tools.

Why the problem is subtle. The two decisions are coupled in opposite directions. For a *fixed* job order, the optimal loading policy is known in closed form—the Keep Tool Needed Soonest (KTNS) rule of Tang and Denardo [32], computable in $O(n|T|)$ time—so the loading subproblem is easy. The difficulty lies entirely in the ordering: a good order places jobs with similar tool requirements next to one another so that few tools change hands, but “similarity” here is a set-overlap relation with no underlying metric, and the capacity constraint couples positions that are far apart in the sequence. The joint problem is NP-hard [11, 20], and even the strongest compact integer formulations [6, 12, 20] become difficult beyond moderate sizes. This structure—an easy inner problem nested inside a hard ordering problem—is what decomposition methods are designed to exploit, and both lines of this work, the structural analysis and the exact methods, rest on it.

The grouping view, and the questions it forces. The SSP is closely tied to a simpler relative, the Job Grouping Problem (JGP), which asks only how few magazine configurations suffice to serve every job. Grouping suggests both the standard heuristic—group the jobs, then order the groups—and a route to exact solution, because a *configuration* (a magazine-filling set of tools) is a natural unit to reason about: the SSP becomes a generalised travelling-salesman problem (GTSP) that visits, in some order, one configuration able to serve each job, paying for the tools that change between consecutive configurations [8]. Two questions follow, and the literature answers neither. First, how good is the heuristic? Group-then-sequence is the method of choice in applications—in colour printing, where the SSP appears verbatim, it comes close to optimal on industrial instances [5], and the strongest metaheuristics seed from JGP solutions [22]—but its record is entirely empirical: how far it can be from the optimum, and when it is exact, has not been analysed. Second, can the SSP be solved exactly through configurations? Branch-and-price works well for the JGP because its set-covering relaxation is tight [27]. Whether the configuration view gives an equally strong exact method for the SSP was open—and the answer turns on whether its natural relaxation can rise above the elementary bound $|U| - b$.

A single quantity links the two. These questions share an answer in that coverage bound $|U| - b$, the number of used tools in excess of the magazine capacity. It is at once the value every configuration relaxation attains and, as the gap analysis shows, the exact optimum on a large part of the instance space. Where it is tight, exact configuration methods close at the root and the heuristic tends to be exact; where it is loose, the heuristic’s gap can open and the same methods branch heavily. That coincidence, developed in Sections 3 and 5, is the connecting thread of this report, and the configuration view that produces it is established once, in Section 2.

Contributions. The principal results are the following.

- *Structural theory of the heuristic gap* (Section 3): the section is withheld from this draft while its proofs undergo the author’s independent verification; the corresponding contributions will be restored with it.
- *Exact methods* (Section 4): a branch-and-Benders-cut solver whose subproblem is the polynomial KTNS oracle; and position-indexed formulations (the position–configuration formulation PCF, its symmetry-reduced variant PCF’, and the position–transition formulation PTF) with a fixed $O(n|T|)$ row set, their pricing subproblems, and an exact branch-and-price.
- *The computational instrument* (Section 5): a complete benchmark of all the methods above against the literature’s compact models, quantifying the coverage bound’s tightness on the standard suites (48.0% among solved instances) and demonstrating mechanically that configuration-based methods—Benders and branch-and-price alike—are bound-limited rather than implementation-limited.

Outline. Section 2 fixes notation, defines the SSP and KTNS, introduces the two-phase heuristic and the configuration–GTSP lens, proves the two structural lower bounds, fixes the switch-cost convention, and reviews the literature. Section 3 develops the structural theory of the heuristic gap; Section 4 presents the exact methods; Section 5 reports the experimental design and the competitiveness assessment; Section 6 concludes; and Section 7 sets out the work remaining in the project. Longer proofs are deferred to Appendix A. A reader pressed for time can follow the spine of the report through the heuristic’s definition (Section 2.3) and the results table of Section 5.2.

2 Preliminaries

2.1 The Job Sequencing and Tool Switching Problem

We are given n jobs $J = \{1, \dots, n\}$, m tools $T = \{1, \dots, m\}$, a magazine capacity $b \in \mathbb{N}$ with $b < m$, and for each job j a set of required tools $T_j \subseteq T$ with $1 \leq |T_j| \leq b$. We write $U = \bigcup_{j \in J} T_j$ for the set of tools used by at least one job.

Definition 1 (Schedule and switch cost). A *schedule* is a pair $(\pi, \mathbf{M})$ consisting of a permutation π of J and a sequence of *magazine states* $\mathbf{M} = (M_1, \dots, M_n)$, $M_i \subseteq T$, satisfying the capacity and feasibility constraints

$$|M_i| \leq b \quad \text{and} \quad T_{\pi(i)} \subseteq M_i \quad (i = 1, \dots, n).$$

With $M_0 = \emptyset$, its *switch cost* is the total number of tool insertions, $\sum_{i=1}^n |M_i \setminus M_{i-1}|$. The SSP asks for a schedule of minimum switch cost; this optimum is denoted Z^* .

Counting insertions is the standard cost model: under a fixed capacity each insertion into a full magazine forces exactly one removal, so insertions and removals differ only by boundary terms, which Section 2.6 makes precise.

Example 1 (A running instance: the 6-ring). We illustrate every definition on one instance, returned to throughout (Figure 1). The 6-*ring* has $n = m = 6$, capacity $b = 3$, and $T_j = \{j, j+1\}$ with indices taken modulo 6 (so $T_6 = \{6, 1\}$); the jobs form a cycle in which consecutive jobs share exactly one tool. Its job–tool incidence matrix (rows are tools 1–6, columns jobs 1–6; a

blank denotes 0) is

	1	2	3	4	5	6
t_1	1					1
t_2	1	1				
t_3		1	1			
t_4			1	1		
t_5				1	1	
t_6					1	1

Process the jobs in the cyclic order 1, 2, 3, 4, 5, 6 and load by the rule of Section 2.2. The magazine evolves as

$$\{1, 2, 3\} \rightarrow \{1, 2, 3\} \rightarrow \{1, 3, 4\} \rightarrow \{1, 4, 5\} \rightarrow \{1, 5, 6\} \rightarrow \{1, 5, 6\},$$

inserting 3 tools to build the first magazine and then one tool at each of positions 3, 4, 5: a switch cost of 6, equivalently 3 once the initial fill is not charged (Section 2.6). Section 3 shows that 6 is optimal, yet the two-phase heuristic of Section 2.3 returns 7. The 6-ring is thus the smallest instance on which grouping-then-sequencing is provably sub-optimal, and it recurs below as the canonical witness of a positive gap.

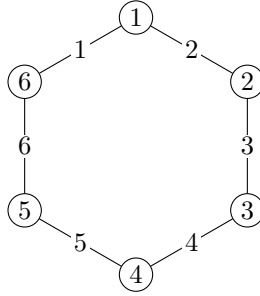


Figure 1: The 6-ring. Jobs 1–6 lie on a cycle; each edge is labelled by the single tool its two endpoints share ($T_j = \{j, j + 1\}$). Adjacent jobs overlap in one tool; non-adjacent jobs are tool-disjoint.

2.2 The tool-loading oracle

The inner problem—optimal loading for a fixed order—is solved by a greedy rule whose optimality is classical.

Proposition 1 (32). *For a fixed permutation π , an optimal magazine sequence is produced by the Keep Tool Needed Soonest (KTNS) policy: whenever a tool must be inserted into a full magazine, evict a tool whose next required use lies furthest in the future (evicting any tool never required again). KTNS runs in $O(n|T|)$ time and returns the minimum switch cost over all loading policies for π .*

A proof appears in Tang and Denardo [32]; see also Crama et al. [11]. KTNS reduces the SSP to a pure sequencing problem, $Z^* = \min_{\pi} \text{KTNS}(\pi)$, and is the exact polynomial subproblem exploited by the Benders algorithm of Section 4.2.

Remark 1 (The loading subproblem is offline optimal caching). For a fixed order the loading problem is exactly offline paging: the magazine is a cache of size b , position i requests the set $T_{\pi(i)}$, and an insertion is a cache miss. Proposition 1 is then the tool-switching form of Bélády’s optimal replacement rule—evict the item whose next request is furthest in the future—which minimises misses on any fixed request sequence [3]. The contrast with the full problem is

instructive: paging fixes the request order, whereas the SSP may *reorder* the jobs, and it is precisely this freedom that turns a polynomially solvable caching problem into an NP-hard one. Every method in this report is, at heart, a way to search over reorderings while delegating the loading to this exact oracle.

2.3 Grouping, sequencing, and the two-phase heuristic

A complementary viewpoint ignores order and asks only which jobs may *share* a magazine.

Definition 2 (Job Grouping Problem). A *group* is a set of jobs $G \subseteq J$ whose combined tool requirement fits in the magazine, $|\bigcup_{j \in G} T_j| \leq b$. The Job Grouping Problem (JGP) partitions J into the minimum number K^* of groups; equivalently, it covers all jobs by the fewest tool configurations of size at most b .

The JGP is itself NP-hard and is solved exactly by the asymmetric-representatives formulation of Jans and Desrosiers [19] or by the set-covering column generation of Crama and Oerlemans [10]. It induces the classical two-phase heuristic for the SSP:

1. solve the JGP to obtain K^* groups and a configuration serving each;
2. solve the *Group Sequencing Problem*—order the configurations by a travelling-salesman-type problem whose transition cost is the number of tools that change, a small instance handled by standard solvers [1, 18]—to obtain a sequence of configurations;
3. concatenate the jobs group by group and evaluate the resulting order with KTNS.

We write H for the switch cost this heuristic returns. Since it produces a feasible schedule, $H \geq Z^*$ always; the objects of study are the *gap* $H - Z^*$ and the *ratio* H/Z^* (Section 3).

Example 2 (Grouping the 6-ring). On the running instance the JGP returns $K^* = 3$; one optimal grouping is $\{1, 2\}, \{3, 4\}, \{5, 6\}$, served by the configurations $\{1, 2, 3\}, \{3, 4, 5\}, \{5, 6, 1\}$. Any two of these three configurations differ in two tools, so every ordering of the groups changes two tools at each of its two transitions, and the best schedule the heuristic produces costs $H = 7$ (free-initial 4). This is one more than the optimum of 6 (free-initial 3) attained in Example 1: a gap of exactly one. The discrepancy is not a failure of the sequencing phase—the groups are ordered optimally—but of the *grouping* phase, which insists on only $K^* = 3$ groups; Section 3 makes this precise.

2.4 The configuration–GTSP lens

Both the heuristic and the exact formulations live in the space of configurations.

Definition 3 (Configurations and clusters). A *configuration* is a set $C \subseteq T$ with $|C| = b$. Let $\mathcal{C} = \{C \subseteq T : |C| = b\}$, and for each job j let $H_j = \{C \in \mathcal{C} : T_j \subseteq C\}$ be the *cluster* of configurations able to serve j . For configurations C, C' put $d(C, C') = |C' \setminus C|$, the number of tools inserted in passing from C to C' .

Restricting to full magazines ($|C| = b$) loses no optimality: retaining a tool that may be useful later never increases cost. Under this view a schedule is a walk $C_{(1)}, \dots, C_{(r)}$ in $\mathcal{C}$ that visits at least one configuration of every cluster H_j , with cost $\sum_l d(C_{(l)}, C_{(l+1)})$; the SSP is therefore a *generalised travelling-salesman problem with overlapping clusters* over $\mathcal{C}$ [8]. The clusters H_j overlap—a rich configuration serves many jobs—which is exactly what distinguishes the SSP from a textbook GTSP with disjoint clusters [14, 26, 28]. That a schedule and such a covering walk have equal cost is immediate in one direction—the distinct consecutive magazines of any schedule form a covering walk of the same cost—and the converse is established as well. It holds in the stronger form ranging over *all* feasible groupings (Section 3, withheld from this draft). This lens recurs throughout: it underlies the gap analysis of Section 3, the position-indexed formulations of Section 4.3, and the reading of the Benders master as a travelling-salesman skeleton (Section 4.2).

2.5 Two structural lower bounds

The following bounds are elementary but pervasive: they drive the gap theory of Section 3, calibrate the linear relaxations of Section 4, and serve as correctness checks for the computational study. Both are stated in the *free-initial* cost (Section 2.6), which does not charge the first magazine fill; the conversion to Definition 1 is the additive constant of Proposition 4.

Proposition 2 (Grouping bound). $Z^* \geq K^* - 1$.

Proof. Fix any schedule and list its magazine states $M_1, \dots, M_n$; collapse each maximal run of equal states to a single representative, obtaining a sequence $C_{(1)}, \dots, C_{(r)}$ in which consecutive entries differ. Every job is served by some state, hence by some $C_{(l)}$, so $\{C_{(1)}, \dots, C_{(r)}\}$ is a family of sets of size at most b covering all jobs. Assigning each job to one configuration that contains its requirement partitions J into groups, one per *distinct* configuration, each group having combined requirement contained in that configuration and hence of size at most b ; this is a feasible grouping, so the number of distinct configurations is at least K^* . Therefore $r \geq K^*$. Finally, each of the $r - 1$ consecutive transitions changes at least one tool, so the free-initial switch cost is at least $r - 1 \geq K^* - 1$. $\square$

Proposition 3 (Coverage bound). $Z^* \geq |U| - b$.

Proof. Each tool of U is required by some job and therefore occupies the magazine at some position, so it is inserted at least once unless it belongs to the initial fill, which contains at most b tools. Hence at least $|U| - b$ insertions occur in the free-initial count. $\square$

Corollary 1. $Z^* \geq \max\{K^* - 1, |U| - b\}$.

The two bounds are individually tight on some families and simultaneously slack on others (Section 3). The coverage bound $|U| - b$ is exactly the value to which the multicommodity-flow relaxation of da Silva and Yanasse [12] collapses—a fact central to the assessment of Section 5.3. The two measure different things: on the 6-ring, $K^* - 1 = 2$ while $|U| - b = 6 - 3 = 3$, so the coverage bound is tight ($Z^* = 3$ free-initial, Example 1) and the grouping bound is slack, whereas on instances built from many nearly tool-disjoint groups the reverse holds. Neither dominates in general, and their maximum, though always valid, is not always attained: over 21,569 enumerated instances with jobs of sizes $\{2, 3\}$ at $b = 3$, 2,410 have Z^* strictly above both bounds, and the 8-job sliding-window ring at $b = 4$ ($T_j = \{j, j + 1, j + 2\}$ modulo 8, $|U| = 8$) has $Z^* = 5 > \max(3, 4)$.

2.6 Switch-cost convention

Formulations in the literature charge the initial magazine fill differently, and comparing their objective values naively produces spurious discrepancies; we therefore fix the convention.

Definition 4 (Empty-start and free-initial costs). The *empty-start* cost charges every insertion from an initially empty magazine ($M_0 = \emptyset$, as in Definition 1). The *free-initial* cost does not charge the first fill; equivalently, it subtracts the $\min(b, |U|)$ tools loaded at the first position.

Proposition 4 (Convention identity). *For every schedule, $\text{cost}_{\text{empty}} = \text{cost}_{\text{free}} + \min(b, |U|)$.*

Proof. An optimal loading (Proposition 1) keeps the magazine full, so the first position loads $\min(b, |U|)$ tools; the empty-start count charges these initial insertions while the free-initial count, by definition, omits them, and every later insertion is counted identically by both. $\square$

The shift is a per-instance constant, so the gap $H - Z^*$ is convention-invariant whereas the ratio H/Z^* is not; ratio statements below are always taken in a single fixed convention. When objective values from different formulations are compared, they are first reduced to the empty-start convention via Proposition 4.

2.7 Related work

Exact formulations. Catanzaro et al. [6] give a family of compact integer formulations with a computational comparison, their strongest models adding overlap and linear-ordering inequalities. Laporte et al. [20] propose an integer program with a branch-and-cut. da Silva and Yanasse [12] give a multicommodity-flow formulation whose linear relaxation equals the coverage bound of Proposition 3. The configuration/GTSP formulation underlying our position-indexed models is developed by Colares and Wagler [8], Colares et al. [9].

Grouping and column generation. Crama and Oerlemans [10] introduce a set-covering formulation of the JGP solved by column generation with a strong relaxation; Jans and Desrosiers [19] give the asymmetric-representatives formulation. The pricing subproblem of the set-covering model is a Set-Union Knapsack Problem [17], NP-hard even under restrictive conditions. For branch-and-price we follow Barnhart et al. [2], Lübbecke and Desrosiers [21], Vanderbeck and Wolsey [33] and branch by the Ryan and Foster [31] same/different rule; subtour elimination uses the constraints of Miller et al. [23] with the lifting of Desrochers and Laporte [13], or lazily separated generalised cuts.

Adjacent exact studies. Two recent theses are close in method. Otiai [27] develops an exact branch-and-price for the Job Grouping Problem with Ryan–Foster branching and finds the set-covering relaxation strong enough to solve benchmark instances in seconds where the compact asymmetric-representatives model times out. Moreira [25] studies the SSP through a magazine-configuration formulation in the sense of Colares and Wagler [8] together with a configuration-generation heuristic, and measures the strength of that formulation’s linear relaxation against the compact models; we revisit those measurements in Section 5.3.

Heuristics. Constructive and metaheuristic upper bounds include the branch-and-bound and bounding ideas of Ghiani et al. [16] and the hybrid genetic search of Mecler et al. [22], which seeds its population from JGP solutions.

The decomposition in practice. The group-then-sequence decomposition is not merely a theoretical device. In colour printing the SSP appears verbatim—ink cartridges are the tools, the cartridge rack the magazine—and Burger et al. [5] solve the industrial problem by exactly this decomposition, modelling the grouping as unicost set covering and the sequencing as a travelling-salesman problem, enumerating optimal groupings by Stirling-number arguments, and measuring the decomposition’s suboptimality empirically: it is small on their instances, consistent with earlier reports they survey. What the literature does not contain is a *worst-case* analysis of the decomposition—bounds on its gap or ratio, extremal instances, or conditions for exactness. To our knowledge no worst-case guarantee has been published for *any* SSP heuristic; the decomposition in particular had only ever been assessed empirically. Section 3 supplies that analysis.

What remains open, and the route taken here. Three observations from the preceding paragraphs set the agenda. First, the decomposition heuristic has a strong empirical record but no worst-case theory; Section 3 builds one. Second, on the exact side the strongest compact relaxation equals the elementary coverage bound $|U| - b$ [12], and the configuration relaxation collapses to the same value on most instances [25]; progress therefore requires either a method that does not lean on the linear relaxation—the Benders route of Section 4.2, whose strength is an exact polynomial subproblem—or a formulation whose relaxation can exceed the bound, the position–transition route of Section 4.3. Third, the configuration view that is so effective for the JGP [27] lacks a tractable exact SSP counterpart, the natural model carrying exponentially

many corrected subtour constraints (Section 4.3); that is the opening the position-indexed formulations address.

3 Structural analysis of the heuristic gap

This section is withheld from the present draft: the author is completing an independent line-by-line verification of its statements and proofs, and the section will be restored in the next revision. Passages elsewhere that depend on its results are marked accordingly; theorem numbering in later sections therefore differs from the full version.

4 Exact methods

The structural analysis of Section 3 (withheld from this draft) shows that $|U| - b$ is often the exact optimum; the question here is whether exact methods can exploit that, and what happens where the bound is loose. Two routes present themselves. Because the loading subproblem is polynomial (Proposition 1), a Benders decomposition can fix the order in a master and settle the loading exactly in the subproblem, generating cuts (Section 4.2). Alternatively the problem can be reformulated over configurations and solved by column generation—but the natural configuration formulation has exponentially many subtour constraints, and making it tractable is the obstacle that the position-indexed formulations of Section 4.3 resolve. We first record the compact formulations that serve as baselines.

4.1 Compact formulations and the coverage relaxation

Three compact formulations from the literature serve as the baselines of the computational study and as the yardstick against which relaxation strength is measured. Catanzaro et al. [6] give a family F1–F5 in job–position and ordering variables, the tighter members adding linear-ordering and overlap inequalities; we use F4 as the baseline because it is the strongest member that scales: its linear relaxation coincides with F3’s at a smaller model size, while F5’s tighter relaxation is so large that at the $n = 40$ benchmark sizes even its root linear relaxation does not complete. Laporte et al. [20] formulate the problem with tool-state variables and a branch-and-cut. da Silva and Yanasse [12] give a multicommodity-flow model in which each tool routes one unit of flow through the position network. The last is singled out by its relaxation.

Proposition 5. *The linear relaxation of the multicommodity-flow formulation equals the coverage bound $|U| - b$ of Proposition 3.*

Established by da Silva and Yanasse [12], and matched exactly by the position formulation’s relaxation with counting rows (Proposition 7), this matters twice: the compact relaxation is no stronger than the elementary counting bound; and since $|U| - b$ equals Z^* on many families (Section 3), the model is fast exactly where that bound is tight and weak where it is not—a dichotomy quantified in Section 5.3.

4.2 A branch-and-Benders-cut algorithm

Idea. A schedule is an order together with a loading. Once the order is fixed, the minimum loading cost is computed exactly and in polynomial time by KTNS; so we place the *order* in an integer master and represent the loading cost by a single continuous variable θ , refined by cuts generated from the exactly solved loading subproblem. In contrast to column generation, whose pricing subproblem for this problem is NP-hard (Section 2.7), the Benders subproblem here is polynomial, so every cut is exact.

Master. Introduce a depot 0 and arc variables $x_{ij} \in \{0, 1\}$ for $i, j \in \{0, 1, \dots, n\}$, $i \neq j$, with $x_{ij} = 1$ when job j is processed immediately after i (the depot marking start and end). The assignment constraints

$$\sum_{i \neq j} x_{ij} = 1 \quad (\forall j), \quad \sum_{j \neq i} x_{ij} = 1 \quad (\forall i),$$

together with subtour-elimination constraints $\sum_{i,j \in S} x_{ij} \leq |S| - 1$ for $S \subseteq \{1, \dots, n\}$, make the x a Hamiltonian order of the jobs. The master is

$$\min \theta \quad \text{s.t.} \quad \theta \geq \sum_{i,j} w_{ij} x_{ij}, \quad \text{assignment, subtour, and Benders cuts,}$$

in which θ lower-bounds the loading cost and the subtour and Benders constraints are separated lazily. The subtour separation is exact by design: violated constraints are detected at integer candidate solutions by connected-component search, so no optimal solution is ever cut off; a fractional separation routine would be an acceleration, not a correctness requirement, and is deliberately absent from the measured configuration.

An initial valid inequality. At the boundary between consecutive jobs i and j at least $w_{ij} := \max(0, |T_i \cup T_j| - b)$ tools are switched: the magazine holds at most b tools yet must serve first T_i and then T_j , so the members of $T_i \cup T_j$ absent at position i —at least $|T_i \cup T_j| - b$ of them—are inserted by position j . Summing over the chosen arcs gives the valid inequality $\theta \geq \sum_{i,j} w_{ij} x_{ij}$, used to initialise the master.

Subproblem. Fix the order and relabel the jobs $1, \dots, n$ along it. With $g_{t,i} \in [0, 1]$ indicating that tool t occupies the magazine at position i and $s_{t,i} \geq 0$ its insertion there, the loading cost is the optimal value of

$$\begin{aligned} \min \quad & \sum_{t,i} s_{t,i} \\ \text{s.t.} \quad & g_{t,i} = 1 & (t \in T_i), \\ & \sum_t g_{t,i} \leq b & (\forall i), \\ & s_{t,i} \geq g_{t,i} - g_{t,i-1}, \quad s_{t,i} \geq 0 & (\forall t, i), \end{aligned}$$

with $g_{t,0} = 0$. The constraint matrix has the consecutive-ones property and is totally unimodular, so the relaxation is integral and, by Proposition 1, its optimum equals KTNS of the order. Writing R_i for the tool set of the job at position i , an optimal dual prices each requirement “tool t present at position i ” by a multiplier $\bar{\alpha}_{t,i}$ and each capacity bound by $\bar{\beta}_i \geq 0$; linear-programming duality then yields the Benders optimality cut

$$\theta \geq \sum_i \sum_{t \in R_i} \bar{\alpha}_{t,i} - b \sum_i \bar{\beta}_i$$

(the multipliers of the unit upper bounds on the $g_{t,i}$ contribute a further term, suppressed here). The requirement multipliers carry the dependence on the order—through which job, hence which R_i , occupies each position—so the right-hand side is linear in the master’s assignment variables; and because the dual feasible region does not depend on the order, the cut is globally valid, the defining property of a Benders cut.

Cut families. Two kinds of cut are separated, and the computational study switches each on and off independently to measure its contribution. (i) *Benders optimality cuts* from the dual above, separated at integer master solutions and, optionally, at fractional ones (*fractional cuts*)

to tighten the bound before branching. (ii) *Combinatorial KTNS cuts*: at an integer order $\bar{\sigma}$ with $\text{KTNS}(\bar{\sigma}) = \bar{z}$, the inequality

$$\theta \geq \bar{z} \left(1 - \sum_{(i,j) \in A(\bar{\sigma})} (1 - x_{ij}) \right)$$

is valid—it forces $\theta \geq \bar{z}$ when the order is exactly $\bar{\sigma}$ (all arcs $A(\bar{\sigma})$ selected) and is vacuous otherwise—and uses only the polynomial oracle, with no dual computation. Finally, *triplet bounds* generalise w_{ij} to ordered triples of jobs, contributing $O(n^3)$ valid inequalities that strengthen the root relaxation.

4.3 Position-indexed formulations

From the configuration model to a polynomial row set. The configuration formulation of Section 2.4, formalised by Colares and Wagler [8], is the natural exact model over configurations. As first stated, however, its subtour elimination is invalid; the correction of Moreira [25], through prize-collecting travelling-salesman constraints, restores exactness at the price of *exponentially many* constraints, one per subset of configurations. A Miller–Tucker–Zemlin reformulation [13, 23] is the standard device for removing such constraints, but here it trades one obstacle for another.

Proposition 6. *A per-configuration MTZ potential is exact but leaves one potential per configuration, hence exponentially many; aggregating the potentials to one per job is polynomial but no longer exact—it can exclude the optimum.*

The aggregated form fails under job domination: if $T_i \subseteq T_j$, every configuration serving j also serves i , so a transition between two such configurations places $\{i, j\}$ at both ends and forces the potentials of i and j into a two-cycle no ordering can satisfy; a four-job instance with $b = 3$ has optimum 1 that the aggregated model values at 2. Neither MTZ variant yields a tractable exact model. The position-indexed formulations below take a different route: they fix, in advance, a row set polynomial in $(n, |T|)$, so column generation only ever adds columns to existing rows, and they carry no subtour constraints at all.

Position–configuration formulation (PCF). Let $y_C^k \in \{0, 1\}$ indicate that configuration C occupies position k ($k = 1, \dots, n$) and let $w_t^k \geq 0$ count the insertion of tool t at position k :

$$\begin{aligned} \min \quad & \sum_{t,k} w_t^k \\ \text{(P)} \quad & \sum_C y_C^k = 1, \quad k = 1, \dots, n; \\ \text{(Cov)} \quad & \sum_k \sum_{C \supseteq T_j} y_C^k \geq 1, \quad j \in J; \\ \text{(W)} \quad & w_t^k \geq \sum_{C \ni t} (y_C^k - y_C^{k-1}), \quad t \in T, k \geq 2; \\ \text{(T)} \quad & \sum_{k \geq 2} w_t^k + \sum_{C \ni t} y_C^1 \geq 1, \quad t \in U. \end{aligned}$$

The configuration variables y are generated as columns; the rows (P), (Cov), (W), (T) number $O(n|T|)$ and never change. Constraint (P) places one configuration per position, (Cov) serves every job, and (W) charges an insertion of t whenever it enters the magazine.

Proposition 7. *PCF is exact: its integer feasible points are exactly the SSP schedules, of equal cost. Its linear relaxation without (T) equals 0; with the counting rows (T) it equals $|U| - b$.*

The vanishing of the plain relaxation is the fractional pathology of Tang and Denardo [32]: a single convex blend of configurations, repeated at every position, satisfies (P), (Cov) and (W) at zero cost. The counting rows (T)—each tool of U is inserted at least once unless present at position one—restore the coverage bound, matching the multicommodity model (Proposition 5) with one extra family of rows.

Position–transition formulation (PTF). To exceed the coverage bound we index *transitions* rather than positions. Augment the configurations with an absorbing tool-less node $\perp$, used to pad schedules that change configuration fewer than $n - 1$ times, and let $z_{C,C'}^k \geq 0$ be a column for the step $k \rightarrow k+1$ that leaves configuration C and enters C' . The formulation is

$$\begin{aligned}
& \min \sum_{k=1}^{n-1} \sum_{(C,C')} d(C,C') z_{C,C'}^k \\
\text{(S)} \quad & \sum_{(C,C')} z_{C,C'}^k = 1, \quad k = 1, \dots, n-1; \\
\text{(F)} \quad & \sum_{(C,C'): t \in C'} z_{C,C'}^k = \sum_{(D,D'): t \in D} z_{D,D'}^{k+1}, \quad t \in T, \quad k = 1, \dots, n-2; \\
\text{(Cov)} \quad & \sum_{k=1}^{n-1} \sum_{(C,C'): T_j \subseteq C} z_{C,C'}^k + \sum_{(C,C'): T_j \subseteq C'} z_{C,C'}^{n-1} \geq 1, \quad j \in J.
\end{aligned}$$

Constraint (S) selects one transition per step; the flow-consistency rows (F) require, tool by tool, that what enters position $k+1$ at the end of step k is what leaves it at the start of step $k+1$, so the columns describe a single coherent walk $C^1 \rightarrow \dots \rightarrow C^n$; and (Cov) serves every job at the configuration of some position—the tail of a step, or the head of the last. The objective charges $d(C,C') = |C' \setminus C|$ per transition. As in PCF the rows number $O(n|T|)$ and are fixed, while the transition columns are generated.

Proposition 8. *PTF is exact, and its linear relaxation is at least $|U| - b$ and can be strictly greater.*

On an instance whose optimum exceeds both bounds of Corollary 1, the PTF relaxation returns 2.10 against a coverage bound of 2; on the 6-ring it is tight at $Z^* = 3$. PTF is thus the first relaxation in this family to improve on the elementary bound, and it does so with a fixed, polynomial row set.

Symmetry reduction: the formulation PCF'. PCF carries a large *padding* symmetry. A schedule using $m < n$ distinct configurations is encoded by repeating configurations to fill all n positions, and the repeats may sit anywhere, so one physical schedule has many equal-cost encodings—distinct nodes to a tree search. We remove this freedom by relaxing the assignment equation (P) to an inequality and forcing the empty positions to a suffix:

$$\text{(P')} \quad \sum_C y_C^k \leq 1 \quad (\forall k); \quad \text{(G)} \quad \sum_C y_C^{k+1} \leq \sum_C y_C^k \quad (k = 1, \dots, n-1);$$

with (Cov), (W), (T) unchanged. Call this formulation PCF'. At an integer point the counts $\sum_C y_C^k$ are non-increasing in k , so the *used* positions form a prefix $1, \dots, p$ and the empties a suffix, with the cost-free first configuration at position 1. The contiguity rows (G) are not decorative: without them an interior empty position would make (W) read the re-entry as a full reload and (T) misattribute the free load, so (G) is what keeps the relaxed model faithful.

Proposition 9. *PCF' is exact, and its linear relaxation equals that of PCF— $|U| - b$ with the counting rows (T), and 0 without. It removes symmetric integer encodings but does not strengthen the bound.*

The proof is in Appendix A. The separation of concerns is deliberate: PCF' attacks symmetry, and only PTF attacks the bound.

Column generation and robust branching. Both PCF' and PTF have $O(n|T|)$ rows but exponentially many columns ($\binom{|T|}{b}$ configurations, or arcs), so their relaxations are solved by *column generation*: a restricted master over a small column pool is reoptimised, and a pricing subproblem repeatedly supplies a column of most-negative reduced cost until none remains. Driving this at every node of a branch-and-bound tree gives *branch-and-price* [2, 21]. Branching on a single column y_C^k is ineffective under the position symmetry and forces the pricer to carry a growing list of forbidden columns. We branch instead on the *presence* variables $a_t^k = \sum_{C \ni t} y_C^k \in \{0, 1\}$ already appearing in (W): fixing a_t^k to 0 or 1 forbids or forces tool t at position k , which in the pricing subproblem merely shifts one dual weight and leaves the oracle's *form* unchanged. This *robust* branching keeps pricing and branching compatible at every node.

Pricing PCF': a b -subset with coverage bonuses. Let $\alpha_k, \gamma_k \leq 0$, $\pi_j, \mu_{t,k}, \lambda_t \geq 0$ be the duals of (P'), (G), (Cov), (W), (T). Direct dual accounting (collecting the coefficient of y_C^k , which enters (W) and (T) through $a_t^k = \sum_{C \ni t} y_C^k$) gives the reduced cost

$$\bar{c}(C, k) = \beta_k - \sum_{t \in C} \rho_t^k - \sum_{j: T_j \subseteq C} \pi_j, \quad \rho_t^k = -\mu_{t,k}[k \geq 2] + \mu_{t,k+1}[k \leq n-1] + \lambda_t[k=1, t \in U],$$

with β_k depending only on the position. For each k the most-negative reduced cost is obtained by maximising the configuration-dependent part,

$$\max_{|C|=b} \sum_{t \in C} \rho_t^k + \sum_{j: T_j \subseteq C} \pi_j,$$

i.e. choosing b tools to collect per-tool rewards plus a bonus for each job whose *entire* requirement lands inside C . This is a Set-Union Knapsack Problem [17]—the grouping-pricing class of Crama and Oerlemans [10]—solved by enumerating the $\binom{|T|}{b}$ subsets at magazine sizes, or by a compact mixed-integer program. The branching dual of a_t^k enters additively in ρ_t^k , so branching does not change the oracle.

Pricing PTF: a coupled pair of subsets. For PTF a column is an arc (C, C') , and its reduced cost contains the bilinear term $\sum_t [t \in C' \setminus C] (1 - \lambda_t)$ together with chaining-dual terms on the tools of C and C' . The pricer cannot therefore separate into a single set: it must choose *two* b -subsets $C \neq C'$ at once, a bilinear selection linearised by McCormick on the products $x'_t(1 - x_t)$ and solved as a mixed-integer program with $2|T|$ binaries. This heavier pricing is the algorithmic price of PTF's stronger bound. The contrast—PCF' pairing a weak bound with a cheap single-set pricer, PTF a stronger bound with a costly coupled-pair pricer—is the central trade-off assessed in Section 5.3.

5 Computational study

Computation in this project is the measuring instrument of the theory: the quantity the theorems single out, the coverage bound $|U| - b$, is also the root relaxation of every configuration method, so a benchmark that records optima and root bounds measures the theory's central variable across the standard instance suites. The campaign also answers a question that predates the theory and motivated the solver in the first place: the SSP has a polynomial loading subproblem—the textbook precondition for Benders decomposition—and whether a decomposition can exploit it against modern compact models had never been tested. The methods of Section 4 were implemented and benchmarked in full; this section reports the design, the results,

and the answer: the decomposition solves exactly as far as its root bound reaches and not one instance further—a structural limit the theory explains, and whose practical half was recovered in full once the master was given the elementary coverage bound.

5.1 Experimental design

The benchmark draws on the standard SSP instance families of Catanzaro et al. [6] ($n = 8\text{--}40$ jobs, $m = 5\text{--}60$ tools, $b = 3\text{--}30$), Crama et al. [11] ($n = 10\text{--}40$, $m = 10\text{--}60$, $b = 4\text{--}30$) and Laporte et al. [20] ($n = 8\text{--}15$, $m = 10\text{--}25$, $b = 4\text{--}20$), spanning a wide range of sizes, tool densities and capacities. Three things are measured. (i) The branch-and-Benders-cut solver is run with its cut families switched on and off in every combination—combinatorial against dual optimality cuts, fractional separation on or off, triplet bounds on or off, eight configurations in all—against the three compact baselines. (ii) The position-indexed branch-and-price is run in its PCF' and PTF variants, on the instances with at most 25 jobs—the prototype's validated range; the compact solvers and the Benders algorithm face the full sets. (iii) For every solver and instance the *root linear-relaxation value* is recorded alongside the optimum where the solver exposes it reliably, so that bound tightness can be studied directly. The common metric is the empty-start KTNS cost of the returned sequence, well defined for every method regardless of its native objective. All runs were executed on the LIMOS SLURM cluster, confined to a homogeneous pool of identical nodes (AMD EPYC 7452, 2×32 cores, 512 GB RAM), so that wall-clock times are comparable across runs. The mixed-integer solver is CPLEX 22.1.1 throughout, pinned to four dedicated threads per run; the branch-and-price prototypes run single-threaded under PySCIPOpt. Each run is allotted 16 GB of memory (32 GB for one memory-heavy configuration, reported where relevant) and a per-instance time limit of 3600s on the primary families and 600s on the secondary ones. All other solver parameters are left at their defaults, so that the comparison measures the formulations rather than parameter tuning. The complete implementation—the solvers, the reimplemented baselines, the campaign harness, and the per-instance result files behind every number in this section—is available at <https://github.com/shankar-n/JGP-SSP-Codes-Shankar>.

Why the secondary suite excludes the flow model. The multicommodity-flow formulation is measured on the primary families only: at the secondary sizes its model is large enough that, under the 600s budget, runs would spend the allowance building rather than solving, adding no information beyond the 3600s primary measurement. Every other method faces both suites.

Protocol: early stop, and what “attempted” means. Within each suite a configuration processes the instances easiest-first and is retired after eight consecutive runs without a proven optimum, since every remaining instance is at least as hard as the ones it just failed. “Easiest” is a fixed difficulty proxy read from the instance header before any run: instances are ordered lexicographically by (n , m , b , matrix density), identically for every method. The number of instances a method *attempted* is therefore itself an outcome of the campaign, not a design choice, and two denominators in the same column of a results table may legitimately differ. A worked example: the entry 1/9 for the Benders solver on the primary suite reads “solved the first (easiest) instance, then failed the next eight in a row, and was retired”—the denominator records where the method died. The budget saved on a failing configuration (up to 3600s times four hundred instances) is spent by configurations still producing information; and since early stop triggers only on failure streaks, a stronger method automatically reaches deeper into its suite.

Validation. Before the campaign, all four exact solvers were checked to reproduce brute-force optima on a bank of small instances (both cut modes of the Benders solver included), and the harness converts every native objective to the common empty-start convention before any comparison. During the campaign the check is global: across the 806 instances solved by at least one method, no two methods ever disagree on the canonical cost.

Scope and limitations. The branch-and-price prototypes are deliberately *unaccelerated*: no column-pool warm start, no dual stabilisation, no heuristic pricing; and branching is the robust scheme on the aggregated variables $a_{t,p}$, which leaves the pricing problem structurally intact (the Ryan–Foster rule solves a problem this master does not have—it exists for set-partitioning masters whose pricers variable branching would destroy). The reason is that the prototype serves here as a measuring instrument for the relaxation itself, and every acceleration would confound the measurement. The results below justify the choice and delimit it: they separate the failures a faster implementation would convert (instances whose optimum already equals the root bound) from the failures no implementation can convert (instances where the bound itself falls short). The engineering agenda—warm columns from the constructive heuristics, heuristic pricing with exact certification, stabilisation—is planned work aimed at exactly the first class.

5.2 Results

Table 1 reports, per method and per instance family, the number of instances solved to proven optimality over the number attempted under the early-stop protocol of Section 5.1. Reading guide: denominators are outcomes (a retired configuration stops attempting); a dash means the protocol retired the configuration before it reached that family; and the counts are mutually consistent in the sense of the validation paragraph above.

Suite	Family ($n; m$)	LSS	CATZ-F4	SSPMF	BBC-LP	PCF'	PTF
primary	Catanzaro A (8–10; 5–10)	51/51	51/51	51/51	24/45	1/1	1/1
	Catanzaro B (15; 18–20)	0/7	0/8	21/36	—	—	—
	Catanzaro C, D (30–40)	—	—	—	—	0/48	0/48
	Crama (10–40)	—	—	—	—	0/48	0/48
	Laporte T7 (10–15)	40/40	40/40	66/78	19/28	—	—
secondary	Laporte T3 (8)	339/339	340/340	n/a	244/318	91/302	71/288
	Laporte T4 (9)	328/330	115/150	n/a	—	2/16	0/2
	Laporte T5 (15)	0/8	—	n/a	—	—	—
totals	primary, of 411	91	91	138	43	1	1
	secondary, of 1,010	667	455	n/a	244	93	71

Table 1: Solved/attempted per method and family under the early-stop protocol; the totals rows use *uniform* denominators (the whole suite), counting every unattempted instance as unsolved, so methods are directly comparable. “—”: the configuration was retired before reaching the family; “n/a”: not run by design (Section 5.1). Time limits 3600s (primary) and 600s (secondary). BBC-LP is shown for the Benders solver *with the corrected master* (coverage row included); after the correction all eight Benders configurations perform within a narrow band (the combinatorial-cut family solves 43/73 primary and 239/318 secondary), whereas before it the best configuration solved 1/9 and 26/48. The branch-and-price columns cover the validated range $n \leq 25$ plus a separate $n \geq 30$ extension run (the Catanzaro C/D and Crama rows), where both prototypes solve nothing.

Compact models. The multicommodity-flow model is the strongest method on the primary families in both dimensions, replicating its state-of-the-art claim [12]. On the 91 primary in-

stances that all three compact models solve, its shifted geometric-mean time is 2.3s against 122s for the 2004 model and 128s for F4—roughly fifty times faster—and it is the only method that penetrates the $n = 15$ block of the Catanzaro suite (21 of 36 attempted there). The largest instance solved by *any* method in the campaign has $n = 15$; no method solves anything at $n \geq 30$. The second observation concerns solver generations rather than formulations: under the same modern CPLEX, the 2004 formulation of Laporte et al. [20] equals F4 on the primary suite (91 solves each, near-identical mean times, the same wall at the $n = 15$, $m \approx 20$ block) and dominates it on the secondary suite: 667 against 455 solves, and twice as fast on the 454 instances both solve (13 against 26s shifted geometric mean). Two of its runs crashed at the default memory allowance, were retried at 32GB, and ended in ordinary timeouts; they are counted as failures. The moral is one of benchmarking hygiene: a comparison against an older formulation that does not state the solver generation measures the solver, not the formulation.

The Benders algorithm. The branch-and-Benders-cut solver has a pure *proof* problem, and the campaign quantifies it exactly: in all thirty time-limited runs of its best configuration, the incumbent held at the time limit already equalled the independently known optimum. The solver finds the answer, then spends the rest of the hour failing to certify it—typically $\sim 250,000$ nodes and $\sim 75,000$ optimality cuts with the dual bound still at its trivial root value, which on capacity-loose instances (where every pairwise bound w_{ij} vanishes) carries no information. The ablation is one-sided: the dual-cut family is the only active axis (the configurations using LP-based dual cuts—BBC-LP in Table 1—solve 26 of 48 attempted secondary instances, the combinatorial-cut family none); triplet bounds are neutral; and fractional Benders cuts are *structurally inactive*—zero violated cuts across the campaign, confirmed in controlled runs with presolve disabled, consistent with the loading subproblem’s value collapsing to the pairwise bound at fractional orderings. The method is bound-limited exactly in the sense of Section 5.3—and the correction that diagnosis dictates, the coverage row $\theta \geq |U|$ in the master, was implemented and re-run in full. The effect is decisive: on the primary suite the solve count rises from 1/9 to 43/73, and on the secondary one from 26/48 to 244/318 for the plain dual-cut arm and 274/374 for its triplet-bound variant, the deepest-reaching configuration; 122 of the dual-cut arm’s 287 solves close at the root node, the median solve time is 0.05s, and the ablation flattens—all eight configurations land within a narrow band, so the earlier dual-cut advantage was an artifact of the missing bound, not a property of the cut families. What does *not* change is equally informative: the corrected solver solves no instance outside the bound-tight class, and in all 104 of its remaining time-limited runs the incumbent again equals the independently known optimum. The Benders approach now solves exactly as far as its root bound reaches, and not one instance further—the residual failure is purely the loose-half bound, which is what the planned window inequalities target. Timing completes the picture: on the 43 primary instances that all four exact methods solve, the corrected Benders’ shifted geometric-mean time is 1.0s against 0.1s for the flow model, 333s for the 2004 model and 616s for F4—within its reach, root closure outruns compact branch-and-bound by two orders of magnitude—and on the 244 commonly solved secondary instances it is on par with the 2004 model (12.9 against 10.7s) and twice as fast as F4 (28.6s), with median solve time 0.05s and 90th percentile 197s. The comparison is conditioned on the instances it solves, which are the bound-tight ones; that conditioning is the diagnosis itself.

The branch-and-price. On the validated range $n \leq 25$, PCF’ proves optimality on 94 of 319 attempted instances and PTF on 72 of 291; in the separate $n \geq 30$ extension run (96 attempts each) both solve nothing. The two prototypes fail differently, and the difference is exactly the relaxation. PCF’ solves where its root bound already equals the optimum (93 of its 94 solves) yet still needs search to *find* the integer solution: median 89 nodes, with only 16% of its solves finishing at a single node—bound tightness and integrality are different obstacles.

PTF, whose relaxation is strictly stronger (Proposition 8), finishes 71 of its 72 solves at a single node: the added bound strength converts tree search into root closures, at the price of a pricing problem heavy enough that it solves fewer instances overall. The failure pattern is the two-regime diagnosis in numbers: of PCF’s 225 failures on instances whose optimum is independently known, 73 lie on *bound-tight* instances—failures of speed alone, the direct target of the engineering agenda of Section 5.1—while 152 lie on bound-loose instances that no acceleration can close without a stronger relaxation.

Bound tightness. Among the 806 instances solved by any method, the coverage bound $|U| - b$ equals the optimum on 387 (48.0%)—an upper-biased estimate, since unsolved instances skew loose; the 419 bound-loose solved instances are exactly the gap testbed used in the planned polyhedral work. The dichotomy of Section 3 is thus not a corner phenomenon: the standard benchmark suites split in half between the regime where the heuristic tends to be exact and exact methods close at the root, and the regime where both must work—which is where the position–transition relaxation and the planned polyhedral work aim.

5.3 The competitiveness of configuration branch-and-price

A guiding question for the exact methods is whether a configuration-based branch-and-price can be made fast for the SSP, and the structural results already constrain the answer. By Propositions 5 and 7 the configuration relaxation collapses to the coverage bound $|U| - b$, and by the gap theory of Section 3 this bound equals Z^* on a large part of the instance space. Where the root bound already equals the optimum branching is unnecessary; where it does not, a relaxation no stronger than $|U| - b$ forces a search tree that faster pricing or branching cannot avoid. The contrast with the Job Grouping Problem is telling: there the set-covering relaxation is strong enough that an analogous branch-and-price closes benchmark instances in a handful of nodes [27], whereas for the SSP the corresponding configuration relaxation does not improve on the trivial bound on most instances [25]. A configuration branch-and-price for the SSP is therefore *bound-limited* rather than implementation-limited. The position–transition formulation, the first in the family to exceed $|U| - b$ (Proposition 8), is the structural lever this diagnosis points to; but it pays off only where the gap opens, and broadening that improvement is part of the planned work (Section 7).

6 Conclusions

6.1 Summary

A single device—the equivalence between the SSP and a generalised travelling-salesman problem over tool configurations—has organised both a structural theory of the standard heuristic and a family of exact methods. The structural side (Section 3) is withheld from this draft pending verification; its summary will be restored with it. On the algorithmic side we gave a branch-and-Benders-cut solver whose subproblem is exact and polynomial, and position-indexed formulations with a fixed row set, among which the position–transition model is the first configuration relaxation to improve on the coverage bound. The methods are implemented, and their large-scale comparison is complete (Section 5.2).

6.2 Beyond the project horizon

The remaining work is set out in Section 7; three further directions fall outside it and are recorded here as lying beyond this project’s horizon.

The position–transition device beyond the SSP. Nothing in the position–transition construction is specific to tool switching except the transition metric: position–transition columns with per-element consistency rows apply to any generalised travelling-salesman problem with overlapping clusters [28]. Whether the relaxation strength observed here persists in that generality is open, and an affirmative answer would carry the contribution beyond the SSP.

Approximation and online variants. The ratio theory of Section 3 invites a systematic worst-case study of the *other* SSP heuristics (constructive and TSP-based), none of which appears to carry a published guarantee; any such comparison must fix the switch-cost convention first. Separately, the caching equivalence of Remark 1 suggests an online SSP, in which jobs arrive over time, analysed competitively against paging machinery—with the offline loading oracle as a natural baseline.

Learning-augmented column generation. Machine-learning acceleration of pricing and branching [4, 15, 24] is an active but crowded field. It is worth pursuing here only if the benchmark data expose a concrete learning signal—such as the pronounced imbalance between zero-gap and positive-gap instances that the structural results predict—rather than as an end in itself.

7 Remaining work

Most of the programme this report set out to complete is done: the structural theory of the heuristic gap (Section 3, withheld from this draft) and the full computational campaigns with their analysis (Section 5). Three items remain. They are independent of one another, and the first two are open research questions rather than pending tasks; no firm dates are attached, as an extended mid-summer break falls within the remaining period.

The exact worst case. Whether the proved quantitative bound at $K^* = 3$ is the true worst case, the $K^* \geq 4$ regime, and the last open case of the $4/3$ ratio—all stated in Section 3 (withheld from this draft). First experiments have concentrated the $K^* = 3$ question at two specific corner regimes.

The polyhedral programme. Two questions drive it: whether odd-ring-like structure is the *only* obstruction to integrality of the JGP covering relaxation, and which facets of the position–transition polytope explain where its bound exceeds $|U| - b$ (Proposition 8). A first scan is complete: the covering relaxation is non-integral on 24% of the $b = 3$ edge family, and non-integrality occurs off-ring (an explicit mixed-size witness has fractional cover 3.5 against $K^* = 4$)—so the obstruction landscape is richer than the odd rings alone, and the dataset for the characterisation question is prepared. The facet study on the small witness instances proceeds from there.

The corrected Benders master. The campaign showed that the Benders master lacked the elementary coverage bound every other method carries (Section 5.2). The correction is a single valid inequality, $\theta \geq |U|$, now implemented and verified. On bound-tight instances the corrected master closes at the root with no Benders cuts at all (on the 6-ring: zero cuts, against 443 before); on bound-loose instances it lifts the root to $|U|$ and search resumes from there. The full re-run with the corrected master is complete, and its outcome is reported in Section 5.2: the solve counts multiply, every remaining timeout still carries an optimal incumbent, and the configuration solves nothing outside the bound-tight class. The item that remains is therefore no longer the re-run but its successor: a bound that reaches into the loose

half. The direction is fixed by an exact computation: solving the pairwise bound to optimality (a travelling-salesman path over the w_{ij} , exact for $n \leq 15$) lifts the root value on a quarter of the bound-loose instances—by 4.9 on average—yet reaches the optimum on under one per cent of them. Pairwise information is thus exhausted, and the next lever is its higher-order form: window inequalities over consecutive job sets S , charging $|\bigcup_{j \in S} T_j| - b$ when the jobs of S run contiguously, generalising the pairwise and triplet rows in one family.

A Proofs

The proofs of the results of Section 3 are withheld with that section.

Proof of Proposition 9. Exactness. Take an optimal schedule and collapse equal consecutive magazines to the sequence of distinct magazines it visits, a trajectory $D_1, \dots, D_m$ ($m \leq n$). Set $y_{D_l}^l = 1$ for $l = 1, \dots, m$, leave positions $m+1, \dots, n$ empty, and $w_t^k = \max(0, a_t^k - a_t^{k-1})$. Then (P') holds ($= 1$ on the prefix, $= 0$ after) and (G) holds (the prefix is contiguous), while (Cov), (W), (T) hold exactly as for PCF; the full-to-empty step $m \rightarrow m+1$ and all later steps add no cost, so the objective is $\sum_{l < m} d(D_l, D_{l+1}) = Z^*$. Hence the optimum is at most Z^* . Conversely, any integer-feasible PCF' point has, by (G), its used positions in a prefix $1, \dots, p$, each holding one configuration D_k ; by (W) the objective is at least $\sum_{k=2}^p d(D_{k-1}, D_k)$, and by (Cov) every job is served by some D_k , so assigning each job to a covering position yields a schedule of cost at most the objective. Hence the optimum is at least Z^* .

LP invariance. Every PCF-feasible point has $\sum_C y_C^k = 1$ for all k and so satisfies (P') and (G); the PCF' relaxation thus minimises over a superset and is no larger than PCF's. For the reverse, summing (T) over $t \in U$ and using $\sum_{t \in U} a_t^1 \leq \sum_{t \in T} a_t^1 = b \sum_C y_C^1 \leq b$ (the last bound now from (P') rather than (P)) gives objective $\geq |U| - b$ at every PCF' point, so the two relaxations coincide wherever PCF's value is $|U| - b$. The plain relaxation, dropping (T), is certified 0 by the same blended configuration used for PCF. $\square$

References

- [1] David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.
- [2] Cynthia Barnhart, Ellis L. Johnson, George L. Nemhauser, Martin W. P. Savelsbergh, and Pamela H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.
- [3] László A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [4] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [5] Alewyn P. Burger, C. G. Jacobs, Jan H. van Vuuren, and Stephan E. Visagie. Scheduling multi-colour print jobs with sequence-dependent setup times. *Journal of Scheduling*, 18(2): 131–145, 2015.
- [6] Daniele Catanzaro, Luis Gouveia, and Martine Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.
- [7] Maria Chudnovsky, Neil Robertson, Paul Seymour, and Robin Thomas. The strong perfect graph theorem. *Annals of Mathematics*, 164(1):51–229, 2006.

- [8] Rafael Colares and Annegret Wagler. Exact formulations for the job sequencing and tool switching problem. *Working Paper, Université Clermont Auvergne, LIMOS*, 2026.
- [9] Rafael Colares, Mauricio de Souza, and Annegret Wagler. The tool switching problem. *Preprint, Université Clermont Auvergne, LIMOS*, 2026.
- [10] Yves Crama and A. G. Oerlemans. A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research*, 78(1):58–80, 1994.
- [11] Yves Crama, Antoon W. J. Kolen, Alwin G. Oerlemans, and Frits C. R. Spieksma. Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems*, 6(1):33–54, 1994.
- [12] Antônio Augusto Chaves da Silva and Horacio Hideki Yanasse. A multicommodity flow model and exact methods for the job sequencing and tool switching problem. *Preprint (compiled 2024); see also Computers & Operations Research*, 2024. SSPMF multicommodity-flow formulation; LP lower bound $M - C$.
- [13] Martin Desrochers and Gilbert Laporte. Improvements and extensions to the Miller–Tucker–Zemlin subtour elimination constraints. *Operations Research Letters*, 10(1):27–36, 1991.
- [14] Matteo Fischetti, Juan José Salazar-González, and Paolo Toth. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. *Operations Research*, 45(3):378–394, 1997.
- [15] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- [16] Gianpaolo Ghiani, Antonio Grieco, and Emanuela Guerriero. Solving the job sequencing and tool switching problem as a nonlinear least cost hamiltonian cycle problem. *Networks*, 55(4):379–385, 2010.
- [17] Olivier Goldschmidt, David Nehme, and Gang Yu. Note on the set-union knapsack problem. *Naval Research Logistics*, 41(6):833–842, 1994.
- [18] Keld Helsgaun. Solving the equality generalized traveling salesman problem using the lin–kernighan–helsgaun algorithm. *Mathematical Programming Computation*, 7(3):269–287, 2015.
- [19] Roel Jans and Jacques Desrosiers. Efficient symmetry breaking formulations for the job grouping problem. *Computers & Operations Research*, 40(4):1132–1142, 2013.
- [20] Gilbert Laporte, Juan José Salazar-González, and Frédéric Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.
- [21] Marco E. Lübbecke and Jacques Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005.
- [22] Jordana Mecler, Anand Subramanian, and Thibaut Vidal. A simple and effective hybrid genetic search for the job sequencing and tool switching problem. *Computers & Operations Research*, 127:105153, 2021.
- [23] Clair E. Miller, Albert W. Tucker, and Richard A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326–329, 1960.

- [24] Mouad Morabit, Guy Desaulniers, and Andrea Lodi. Machine-learning-based column selection for column generation. *Transportation Science*, 55(4):815–831, 2021.
- [25] Bernardo Bastos Pereira Moreira. A new combinatorial approach to solve the job sequencing and tool switching problem. Master’s thesis, Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Brazil, 2026.
- [26] Charles E. Noon and James C. Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR: Information Systems and Operational Research*, 31(1):39–44, 1993.
- [27] Felipe Otiai. Flexible manufacturing systems: Job grouping and tool management. Master’s thesis, Federal University of Minas Gerais, 2026.
- [28] Petrică C. Pop, Ovidiu Cosma, Cosmin Sabo, and Corina Pop Sitar. The generalized traveling salesman problem: A comprehensive review. *European Journal of Operational Research*, 314(3):819–835, 2024.
- [29] Caroline Privault and Gerd Finke. Modelling a tool switching problem on a single nc-machine. *Journal of Intelligent Manufacturing*, 6(2):87–94, 1995.
- [30] Caroline Privault and Gerd Finke. K-server problems with bulk requests: An application to tool switching in manufacturing. *Annals of Operations Research*, 96(1/4):255–269, 2000.
- [31] D. M. Ryan and B. A. Foster. An integer programming approach to scheduling. *Computer Scheduling of Public Transport*, pages 269–280, 1981.
- [32] Christopher S. Tang and Eric V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5):767–777, 1988.
- [33] François Vanderbeck and Laurence A. Wolsey. Reformulation and decomposition of integer programs. In Michael Jünger et al., editors, *50 Years of Integer Programming 1958–2008*, pages 431–502. Springer, Berlin, Heidelberg, 2010.